PRD — Sherlock's Last Case- v5

Master Specification, v5

1. Overview

Sherlock's Last Case is a browser-based, team-play detective game for a college technical festival, with two parallel gameplay tracks.

- **SQL Track:** 10 cases, `SQL01`–`SQL10`. Players investigate using a real, restricted SQL console.
- **Logic Track:** 10 cases, `LOG01`–`LOG10`. Players investigate through an Evidence Browser with search, filtering and sorting. An optional SQL console is available but never required.

Both tracks use the same underlying game engine for teams, phases, investigation threads, hints, scoring, evidence unlocks and final deductions. The story content and evidence-access experience differ.

Team size: 1–3 players, comprising one team head and up to two investigators.

Assignment: Teams register first. Immediately before gameplay, organizers manually assign each registered team its track and specific case. The team head then enters the assigned case code, case password and registration credential to enter the game. Teammates join using the generated team code.

Technology: Next.js, Supabase/Postgres and Vercel.

Expected scale: Approximately 30–35 concurrent computers and 10–12 teams.

Target duration: 15–20 minutes per case on average, with around 10 minutes for strong players as an initial goal. Actual timings must be measured during playtesting.

2. Goals and Non-Goals

Goals

- Create genuinely engaging investigations for both SQL-experienced and SQL-new players.
- Make every case solvable through its intended interface without requiring knowledge of the other track.
- Prevent cross-team and cross-case evidence leakage.
- Enforce deterministic, server-side grading.
- Support organizer-controlled assignment immediately before gameplay.
- Provide shared team progress, hints, scoring and evidence unlocks.
- Recover correctly after browser closure, disconnection or refresh.
- Provide separate live leaderboards for the two tracks.
- Enforce defense-in-depth protections on SQL execution.
- Support solo, duo and three-player teams without making any thread depend on having three players.

Non-Goals

- Persistent player accounts beyond the event.
- More than 20 case templates.
- AI-graded answers or free-text interpretation.
- Cross-event player histories.
- Requiring SQL knowledge to complete a Logic-track case.
- Player-controlled case or track switching after assignment.

---

3. Track and Case Model

Case registry

`cases`

- `id`
- `case_code` — `SQL01`–`SQL10` or `LOG01`–`LOG10`
- `game_track` — `SQL` or `LOGIC`
- `title`

- `hook_text`
- `status` — `draft`, `tested`, `published`, `disabled`
- `target_duration_minutes`
- `content_version`

Every case has its own cast, story, evidence, investigation threads, questions, hints, answer keys and scoring configuration.

Case catalogue

All titles and story content remain subject to change during authoring.

SQL Track

Code	Working title	Status
SQL01	Seven Minutes Late	TBD
SQL02	The Last Show	TBD
SQL03	TBD	TBD
SQL04	TBD	TBD
SQL05	TBD	TBD
SQL06	TBD	TBD
SQL07	TBD	TBD
SQL08	TBD	TBD
SQL09	TBD	TBD
SQL10	TBD	TBD

Logic Track

Code	Working title	Status
LOG01	TBD	TBD
LOG02	TBD	TBD
LOG03	TBD	TBD
LOG04	TBD	TBD
LOG05	TBD	TBD
LOG06	TBD	TBD
LOG07	TBD	TBD
LOG08	TBD	TBD

Code	Working title	Status
LOG09	TBD	TBD
LOG10	TBD	TBD

These are working titles, not approved story specifications. All 20 cases must pass the authoring and validation requirements in Section 6 before publication.

---

4. Registration and Organizer-Controlled Assignment

4.1 Registration happens first

Participants register before gameplay. Registration records do not require a case assignment at the time of registration.

`registrations`

- `id`
- `registration_ref` — unique reference or credential for the registered team
- `team_ref`
- `team_size`
- `registration_status`
- `created_at`

Registration data must not contain an assumed final case assignment.

4.2 Manual assignment immediately before gameplay

Just before the event begins, an authorized organizer uses the admin dashboard to assign each registered team:

1. A track: `SQL` or `LOGIC`.
2. A specific published case belonging to that track.

The organizer selects both values manually. The system validates that the selected case belongs to the selected track.

`team_case_assignments`

- `id`
- `registration_id`
- `case_id`

- `assigned_by`
- `assigned_at`
- `status` — `assigned`, `started`, `completed`, `cancelled`

Rules:

- Each registered team can have only one active assignment.
- Assignment must be completed before the team starts playing.
- A registered team without an assignment cannot enter gameplay.
- The assigned case must be published and enabled.
- The assignment is fixed once the game starts.
- Before gameplay starts, an organizer may correct an assignment.
- After gameplay starts, reassignment requires an explicit administrative reset and must be logged.
- Players cannot change their assigned track or case by altering browser requests.
- Multiple teams may receive the same case.
- Case allocation may be balanced manually by organizers, but the system must not require every case to be assigned.

4.3 Case credentials

Each published case has a case code and a case password managed by the organizers.

The team head enters:

- Their registration credential.
- The assigned case code.
- The case password.

The server verifies that:

1. The registration exists and is eligible.
2. The organizer has assigned a case to that registration.
3. The submitted case code matches the assigned case.

4. The password is valid for that case.
5. The case is published and available to start.
6. No conflicting active team session already exists for that registration.

The case code and password help identify and validate the assigned case. They do not grant permission to switch to another case.

Passwords must be stored securely, never exposed in client-side source code or logs, and compared using a server-side password-verification mechanism.

The team head then creates the team session and receives a unique team code. Investigators join through that team code.

4.4 Assignment integrity

The registration-to-case assignment must remain stable across retries and reconnects.

If the head submits the same valid join request twice, the server must not create two independent teams for the same registration. Use transactional creation and idempotency protection.

If a registered team is reassigned before starting, the new assignment must be reflected in the join flow. If a team has already started, the assignment cannot change without an authorized reset.

---

5. Team and Session Model

5.1 Team structure

- One head and zero to two investigators.
- A team may start and finish with one player.
- A fourth player is rejected.
- One head per team.
- Team codes must be unique.
- All team members share the same case, score, hints, unlocked evidence and progression.

The team head may assign investigation threads among available teammates, but thread availability and completion requirements must never depend on

team size.

5.2 Joining flow

First join:

`POST /api/join`

The head submits the registration credential, assigned case code and case password.

The server validates the registration and assignment, creates the team and head membership, records the started assignment, and returns a signed session token and team code.

Subsequent joins:

The investigator submits the team code. The server verifies that the team exists, accepts new members only if fewer than three players are active members of that team, and issues a player session.

5.3 Session authority

The server is the sole authority for:

- `team_id`
- `player_id`
- `case_id`
- `game_track`
- player role
- current phase
- unlocked threads and evidence
- task completion
- hint usage
- score
- game completion status

The browser may display this state but cannot set it.

Every request must derive identity and assignment from the verified server session and database records rather than trusting client-supplied IDs.

5.4 Recovery

A browser refresh, temporary disconnection or closed tab must not reset progress.

On reconnect, the server restores:

- assigned case and track
- team membership and role
- current phase
- thread and task completion
- unlocked evidence
- hints already used
- answer-attempt history
- score
- final-deduction progress

The server remains authoritative; browser cache is never used to restore scores or unlocks.

---

6. Game Flow and Case Authoring

6.1 State machine

`WAITING → PHASE_1 → PHASE_2 → PHASE_3 → FINAL_DEDUCTION → FINISHED`

All transitions are server-side effects of satisfying the relevant case's requirements.

The client cannot directly set the current phase or mark a thread complete.

6.2 Phase purposes

Phase	Purpose	Typical content
Phase 1 — Discovery	Understand the incident and identify promising leads	Broad evidence, access records, initial statements and early contradictions
Phase 2 — Investigation	Test theories and investigate people, objects or events	Large character dossiers, cross-table comparisons, alibi checks and new leads

Phase	Purpose	Typical content
Phase 3 — Reconstruction	Connect the evidence and establish responsibility	Reinterpreted earlier clues, independent evidence streams and difficult deductions
Final Deduction	Submit the final explanation	Responsible character and explanation of what happened

The phases have different investigative purposes. They must not simply be easy, medium and hard versions of the same question pattern.

6.3 Investigation threads

`investigation_threads`

- `id`
- `case_id`
- `phase_number`
- `subject_character_id` — nullable when the thread concerns an object, event or location
- `intro_text`
- `outcome_type`

`thread_evidence_tables`

- `thread_id`
- `table_name`
- `unlock_condition`

`team_thread_progress`

- `team_id`
- `thread_id`
- `status`
- `completed_at`

A thread is a focused investigation into a person, object, event, location or theory.

A typical thread contains 3–5 connected questions following a progression such as:

Establish → Compare → Contradiction → Connection → Decision

The exact number and structure of questions can vary by case.

Thread outcomes may include:

- `CLEARED`
- `LEAD_DISCOVERED`
- `REMAINS_PERSON_OF_INTEREST`
- `COMPLETED`

Outcomes are determined by the story and its evidence. Not every thread should be a dead end. Some clear innocent people, others establish motive, reveal hidden relationships or open further investigations.

Thread count and mandatory completion requirements are defined per case, independently of team size. Solo players must be able to complete all required investigations sequentially.

All evidence and thread unlocks are evaluated server-side.

6.4 Question design rules

Every case must follow these principles:

1. Questions should ask players to make investigative moves, not merely retrieve isolated values.
2. Each case must have a connected evidence spine designed before its questions.
3. Character dossiers should generally contain 30–100+ rows where appropriate, mostly ordinary records rather than an obvious culprit table.
4. Important evidence should gain meaning when compared with other records.
5. At least two or three Phase 1 clues should change meaning when revisited in Phase 3.
6. Each case must have a distinct central detective trick.
7. Characters should have believable lives, relationships, pressures and reasons for their actions.
8. Include 2–4 realistic red herrings with genuine innocent explanations.

9. Vary motives and evidence structures across the 20 cases.
10. Use Indian names and plain, everyday English in every player-facing prompt.
11. Avoid making every mystery financial or using the same missing-alibi structure repeatedly.
12. Frame the final question as: **"Who was responsible for what happened to [victim]?"** This accommodates accidents, concealment and unintended consequences as well as deliberate harm.
13. Do not reveal the intended solution through table names, column names or obvious record labels.
14. Every conclusion must be uniquely supported by the available evidence.

6.5 Evidence access

Each case defines:

- evidence available at the start
- evidence unlocked by each thread or answer
- required threads for each phase transition
- optional threads
- evidence that must remain hidden until a defined point
- story events triggered by successful deductions
- final-deduction prerequisites

A table may contain multiple evidence types, but the server must enforce the case's intended access permissions for every interface.

6.6 Case publication checklist

Before publication, each case must pass:

- Answer-key validation against the actual seeded database.
- A check that every required thread can be completed by a solo player.
- A check that the intended solution follows from the evidence alone.
- A check that red herrings have plausible innocent explanations.
- A check that both evidence interfaces respect the same unlock rules.

- A check that no question is answerable solely by guessing an obvious table or column name.
- A check that the complete scoring configuration totals exactly 100 base points.
- A playtest by someone unfamiliar with the intended solution.

---

7. Scoring, Hints and Attempts

7.1 Score budget

Every case has a maximum base score of exactly 100 points.

The sum of all question base values and both final-deduction component values must equal 100.

Hints and incorrect-answer penalties reduce the score earned; they do not add to the maximum.

7.2 Hint scoring

For an ordinary 10-point question:

Hints used	Reward for a correct answer
0	10
1	8
2	6
3	4
4	2

Each question declares its own base value and hint costs. Questions with a different base value must use an explicitly defined reward schedule.

Hints are shared across the team. If one teammate opens a hint, it is visible to all teammates and its cost applies to the team's shared reward for that question.

A team cannot regain points by repeatedly opening and closing a hint or resubmitting the same correct answer.

7.3 Incorrect answers

- Every incorrect committed answer costs 2 points.

- SQL execution errors and unsuccessful exploratory queries cost nothing.
- Incorrect answers may be retried unless a case explicitly defines otherwise.
- The server records each committed attempt.
- The score cannot fall below zero.
- A correctly completed question or final component awards its points only once.
- Replaying a successful submission must not award points again.

7.4 Task attempts versus task completion

`task_attempts` stores the complete history of submitted answers.

`team_task_completions` stores the successful completion of each task.

`task_attempts`:

- `id`
- `task_id`
- `team_id`
- `player_id`
- `submitted_value`
- `is_correct`
- `hints_used`
- `points_awarded`
- `idempotency_key`
- `submitted_at`

`team_task_completions`:

- `team_id`
- `task_id`
- `completed_by`
- `completed_at`
- `points_awarded`

Enforce one completion per `(team_id, task_id)`. Do not impose a uniqueness constraint that prevents multiple legitimate attempts from being recorded.

The answer submission, attempt record, completion record, score update and any triggered state transition must be processed transactionally.

An idempotency key prevents the same request from being processed twice. A new intentional retry must use a new key.

7.5 Score validation

Before publication, validate that the base values of all graded components sum to exactly 100.

The server calculates all points. The browser never sets or modifies scores.

---

8. Final Deduction

Every case has two final-deduction components.

Component A — Responsibility

Question: Who was responsible for what happened?

The team selects a character from that case's roster.

Component B — Explanation

Question: What best explains what happened?

The team selects from 3–4 author-written options, with one correct answer.

Both components are deterministic. There is no free-text interpretation or AI grading.

`final_attempts`

- `id`
- `team_id`
- `component` — `culprit` or `mechanism`
- `submitted_value`
- `is_correct`
- `idempotency_key`
- `submitted_at`

Final-attempt rules

- Incorrect submissions may be retried.
- Each incorrect committed attempt costs 2 points.
- A component awards its points only on its first correct resolution.
- The server stores every attempt.
- Correctly resolving one component does not require resubmitting it to earn points.
- The team can finish only when both required components are correctly resolved and all other mandatory completion conditions are satisfied.

The case configuration assigns explicit base points to each final component. For example, 30 points for responsibility and 20 points for the explanation leaves 50 points for the investigation questions. This is an example allocation, not a mandatory split.

All components combined must have a base maximum of exactly 100 points.

---

9. Dual Evidence Interface

9.1 SQL Track

The SQL console provides:

- a SQL editor
- query results grid
- row limit and truncation notice
- friendly query errors
- query execution feedback

The SQL console is the primary investigation interface for the SQL track.

9.2 Logic Track

The Evidence Browser provides:

- access to unlocked tables
- search
- filtering

- sorting
- pagination
- comparison of records across separate tables by inspecting each table independently

There is deliberately no visual join builder or relationship tool. Players must connect evidence themselves.

The optional SQL console is available from the beginning of a Logic case. It is never required for progression, and using it does not change the score or unlock rules.

9.3 Shared evidence access

Both interfaces must enforce the same evidence permissions and unlock conditions.

The SQL console must not reveal locked tables through queries, metadata or database catalog access.

The Evidence Browser must not reveal locked rows or tables through alternate endpoint parameters.

The Evidence Browser uses parameterized server-side queries. The browser must not submit arbitrary SQL through the Logic interface.

Both interfaces are built and tested early, before the main load test.

---

10. Non-Functional Requirements

Scale

Approximately 30–35 concurrent clients and 10–12 teams.

Availability

The application must operate unattended except for administrative intervention during the approximately 1–2 hour event window.

Performance targets

- SQL query and Evidence Browser response: approximately 1 second under normal conditions.

- Realtime phase and thread updates: approximately 1 second under normal conditions.
- Session restoration: approximately 2 seconds under normal conditions.

These are targets to validate, not unconditional guarantees.

Timing

The 15–20 minute case duration is a goal. Actual completion time must be measured using first-time players from both tracks. Thread count, dataset size, question difficulty and hint availability should be adjusted per case rather than forcing every case into an identical structure.

---

11. Technical Architecture and Security

11.1 SQL execution security

Use layered protection:

1. A dedicated Postgres role with SELECT-only grants and no write access.
2. Row-Level Security on every evidence table, scoped to the authenticated team's assigned case.
3. Statement timeout of approximately 3–5 seconds.
4. Enforced row limits and truncation.
5. Per-player query rate limiting.
6. Client-side syntax checks only for user experience, never as a security boundary.

The SQL execution role must be a normal RLS-subject role, not the Supabase service-role key.

The query service must reject multiple statements and prevent access to system catalogs or unrelated schemas where not required. Database permissions remain the load-bearing protection even if application validation fails.

11.2 Evidence Browser security

The Evidence Browser uses parameterized server-side queries against authorized evidence.

The server verifies:

- team session
- assigned case
- track permissions
- table unlock state
- requested filter and sort fields
- pagination limits

Only explicitly allowed columns and operations may be queried.

11.3 Connection handling

Use pooled Supabase/Postgres connections suitable for the deployment environment. Do not connect directly from the browser to Postgres.

Secrets remain in server-side environment variables.

11.4 Transactional consistency and idempotency

The following operations must be transactional:

- team creation and assignment activation
- task attempt recording and scoring
- successful task completion
- thread and phase progression
- final-deduction attempts and scoring
- team completion

Duplicate requests must not duplicate teams, completions, score awards or state transitions.

11.5 Realtime with authoritative fallback

On page load, fetch authoritative state before or while subscribing to the team's realtime channel.

Realtime events update the displayed state, but correctness never depends on an event arriving.

After a disconnect, reconnect and fetch authoritative state again.

Use one logical channel per team.

11.6 Logging

`query_logs`

- `id`
- `team_id`
- `player_id`
- `query_hash`
- `execution_time_ms`
- `row_count`
- `success`
- `error_type`
- `created_at`

Logs are retained during the event for debugging and operational review, then discarded according to the event's data-retention policy. Do not log passwords, session tokens or full sensitive credential values.

---

12. Data Model

Core tables

`cases`
`registrations`
`team_case_assignments`
`teams`
`players`
`case_phases`
`characters`
`investigation_threads`
`thread_evidence_tables`
`tasks`
`task_hints`
`task_attempts`
`team_task_completions`
`team_thread_progress`

`team_progress`

`team_scores`

`final_attempts`

`story_events`

`query_logs`

Required relationships and constraints

- `cases.case_code` is unique.
- `registrations.registration_ref` is unique.
- A registration can have at most one active case assignment.
- A team can have at most one active case assignment.
- Each team has exactly one case once its game starts.
- Each team has at most one head.
- A team has no more than three players.
- `team_task_completions` has a unique constraint on `(team_id, task_id)`.
- Attempt-history tables allow multiple submissions.
- Every team-progress record references the assigned case's content.
- Every case-specific evidence row carries `case_id` or is accessed through a strictly scoped case-specific schema.
- Every evidence access path enforces case and unlock permissions.

Assignment relationship

The registration is assigned before gameplay. The team session is created when the head successfully joins.

The assignment must be linked to the resulting team in the same transaction that activates the session. This avoids requiring a team ID before the team exists.

Scores

`team_scores`

- `team_id`
- `points_by_component`

- `total_points`
- `updated_at`

The server maintains score totals from authoritative attempts and completion records. Client-provided score values are ignored.

Case evidence

Each case has its own evidence tables, such as access logs, messages, delivery records, character dossiers and inventory histories.

All evidence access is restricted to the assigned case. Table names, columns and seed data must be validated before a case is published.

---

13. API Surface

Endpoint	Purpose
`POST /api/join`	Head: registration credential + assigned case code + password. Investigator: team code.
`POST /api/query`	Execute a restricted SQL query for an authorized team
`POST /api/evidence`	Search, filter, sort and paginate an unlocked Logic-track evidence table
`POST /api/submit-task`	Record an answer attempt, update score and process valid progression
`POST /api/submit-final`	Submit or retry a final-deduction component
`GET /api/state`	Return authoritative team, assignment, phase, thread and score state
`GET /api/leaderboard?track=SQL`	Return SQL leaderboard
`GET /api/leaderboard?track=LOGIC`	Return Logic leaderboard
`POST /api/admin/assign-case`	Assign or change a registered team's case before start, subject to authorization and state rules
`POST /api/admin/reset-team`	Reset a team and explicitly authorize reassignment when needed
`GET /api/admin/dashboard`	Return live operational view across both tracks

Every endpoint must authenticate the session or organizer, validate authorization, enforce assignment and evidence permissions, and return only the minimum required information.

Admin endpoints must not be accessible to player sessions.

---

14. Admin Dashboard

Registration and assignment

- View registered teams and team sizes.
- Assign each team a track and case immediately before gameplay.
- Validate that the selected case belongs to the chosen track.
- Display assignment status: unassigned, assigned, started, completed.
- Detect missing assignments before the event starts.
- Change an assignment before the game begins.
- Require an explicit reset for changes after gameplay starts.
- Record who made each assignment or reassignment.

Live gameplay monitoring

Display:

- team and registration reference
- track and case code
- current phase
- active investigation threads
- hints used
- score
- elapsed time
- connected players
- completion status

Administrative actions

- Reset a team.
- Inspect submitted answers and attempts.
- Inspect recent SQL errors.

- Pause or resume gameplay.
- Reassign a team when allowed.
- Export team assignments and leaderboards.

Leaderboards

Provide separate SQL and Logic leaderboards.

A combined overview may show participation and completion, but must not imply that raw scores across the tracks are directly comparable unless the scoring and difficulty have been validated.

Tie-break

Within each track, ties are broken by the earliest server timestamp at which the team correctly resolved its final required deduction component.

---

15. Build Roadmap

Stage	Goal
0	Foundations: repository, deployment skeleton, environment configuration and core database schema
1	Registration, manual assignment model, head/investigator join flow and session recovery
2	Shared game engine: phases, threads, attempts, completions, scoring and idempotency using stubbed content
3	SQL console and Evidence Browser built together, tested against the same case and unlock permissions
4	Progressive hints, scoring validation, final-deduction retries and leaderboards
5	Realtime synchronization with authoritative fallback
6	Organizer assignment interface, dashboard, reset and recovery controls
7	Load testing and adversarial testing across both interfaces, approximately 35 clients
8	Author and seed all 20 cases; validate answer keys, evidence unlocks and scoring
9	Full event-day rehearsal and real playtests measuring completion time for both tracks

Stage 0 acceptance criteria

Before moving forward:

- The application deploys successfully.
- Server-side environment variables and database connectivity are verified.
- Core schema migrations run cleanly.
- The database distinguishes registrations, assignments and active teams.
- The session and authorization design is documented.
- No player-facing endpoint can bypass assignment validation.

Stage 1 acceptance criteria

- A registration can exist without an assignment.
- An organizer can assign a track and case immediately before play.
- An unassigned registration cannot start.
- The head's credentials are checked against the registered team's assignment.
- A valid first join creates exactly one team and one head.
- Investigators can join with the team code.
- Repeated join requests do not create duplicate teams.
- A reconnect restores the same assignment and state.

Stage 2 acceptance criteria

- A task attempt is recorded exactly once per idempotency key.
- Intentional retries are stored separately.
- A successful task is completed only once.
- Points are awarded only once for a successful completion.
- Hint costs and incorrect-answer penalties are applied transactionally.
- Phase and thread transitions cannot be set by the client.
- Concurrent submissions cannot produce duplicate score awards.

Stage 3 acceptance criteria

- Both interfaces read the same authorized evidence.
- Neither interface can access locked tables or unrelated cases.
- The Logic track is playable without SQL.
- Optional SQL access does not grant additional progression privileges.
- Pagination, row limits and filter validation are enforced server-side.

Stage 7 acceptance criteria

Test:

- 30–35 concurrent clients.
- Multiple teammates submitting answers simultaneously.
- Duplicate and out-of-order requests.
- Incorrect answer retries.
- Hint usage by different teammates.
- Session recovery and reconnects.
- Cross-case access attempts.
- Locked evidence access attempts.
- SQL injection and unauthorized write attempts.
- Organizer resets and assignment changes.
- Leaderboard correctness.

---

16. Resolved Decisions Log

- Two tracks: 10 SQL cases and 10 Logic cases.
- Organizers manually assign each registered team a track and case immediately before gameplay.
- Registration does not require an immediate case assignment.
- The team head uses a registration credential, the assigned case code and case password to join.
- The server verifies that the entered case matches the organizer's assignment.

- Investigators join using the team code.
- Teams contain 1–3 players, with no minimum.
- A team may complete all required threads alone.
- Logic-track SQL access is optional and available from the start.
- Both interfaces use the same evidence permissions and unlock rules.
- Each case has a maximum base score of 100, including both final components.
- Ordinary 10-point questions award 10/8/6/4/2 according to hints used.
- Incorrect committed answers cost 2 points; SQL execution errors are free.
- Incorrect answers, including final-deduction answers, can be retried.
- Every attempt is recorded; successful completion and scoring occur only once.
- Thread outcomes are determined by the story rather than a forced dead-end pattern.
- The 15–20 minute target must be validated through playtesting.
- Case content remains open for authoring; all 20 cases must pass the publication checklist.
- Leaderboards are separate by track.
- Query logs are retained during the event and discarded afterward according to the retention policy.
- Case reassignment after gameplay starts requires an authorized reset.
- The server is authoritative for assignment, evidence access, progression, hints and scores.

Final implementation principle

Build one secure, configurable investigation engine, then populate it with 20 distinct cases.

Registration, assignment, sessions, evidence access, thread progression, hints, attempts, scoring and final deductions must work reliably before all the story content is authored.

The SQL and Logic tracks should feel different to play, but they should not require separate implementations of the core game engine.